

PORTADA

Técnicas de Programación

Unidad 3 — Técnicas de Programación: Código Limpio

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Escribes código una sola vez. Alguien —tú mismo en seis meses, o un compañero de equipo— lo va a leer muchas más veces de las que tú lo escribiste.

"Indeed, the ratio of time spent reading versus writing is well over 10 to 1." — Robert C. Martin, *Clean Code* (2008)

TRANSICIÓN

De la estructura al detalle

SOLID te ayuda a decidir **cómo dividir** responsabilidades entre clases. El **código limpio** opera un nivel más abajo: dentro de cada clase y cada método, ¿el código se entiende con solo leerlo, o hace falta adivinar? Hoy trabajas las prácticas que hacen esa diferencia: nombres, funciones pequeñas, comentarios que valen la pena y formato consistente.

CONCEPTO

¿Qué es código limpio?

Código limpio es aquel que **se puede leer y modificar sin necesitar preguntarle al autor**. No es una cuestión estética: código difícil de entender es más lento de mantener y más propenso a errores cuando alguien lo cambia sin captar todas sus implicaciones. Cuatro frentes concretos donde se nota: **nombres**, **funciones**, **comentarios** y **formato**.

CONCEPTO

Nombres que dicen lo que hacen

Un nombre bien elegido responde por sí solo tres preguntas: **qué contiene** (variables), **qué hace** (métodos) y **qué representa** (clases). Si necesitas un comentario al lado para explicar un nombre, el nombre está mal elegido.

- Variables: sustantivos claros — `listaEstudiantes` , no `l` o `datos` .
- Métodos: verbos que describen la acción — `calcularPromedio()` , no `procesar()` .
- Evita abreviaturas ambiguas (`calc` , `tmp` , `d`) salvo convenciones muy conocidas.

CONTRA EJEMPLO

Nombres que obligan a adivinar

```
public double calc(double d, int t) {  
    if (t == 1) return d * 0.05;  
    if (t == 2) return d * 0.10;  
    return 0;  
}
```

Nadie puede saber qué calcula `calc`, qué representa `d`, ni qué significan los valores `1` y `2` de `t` sin leer el cuerpo del método (o preguntarle al autor).

```
public double calcularDescuento(double montoCompra, TipoCliente tipoCliente) {  
    if (tipoCliente == TipoCliente.VIP) return montoCompra * 0.05;  
    if (tipoCliente == TipoCliente.FRECUENTE) return montoCompra * 0.10;  
    return 0;  
}
```



CONCEPTO

Funciones pequeñas, que hagan una sola cosa

Una función limpia es corta (idealmente cabe en pantalla sin hacer scroll) y hace **una sola cosa** — la misma idea de responsabilidad única, aplicada a métodos en vez de a clases. Señales de que un método hace demasiado: tiene varios niveles de indentación anidados, mezcla validación con lógica de negocio, o su nombre necesita la palabra "y" para describirlo ("*valida y guarda y notifica*").

CONCEPTO

Parámetros: menos es más claro

Un método con más de tres parámetros es difícil de leer en el punto donde se llama, y fácil de invocar con el orden equivocado. Un caso especialmente confuso son los **parámetros booleanos como bandera** (*flag*): obligan a leer la firma del método para saber qué hace `true` .

```
// Poco claro en el punto de uso
generarReporte(estudiante, true, false, true);
```

```
// Mejor: métodos con nombres específicos, o un objeto de configuración
generarReporteDetallado(estudiante);
```

CONCEPTO

Comentarios: la última línea de defensa, no la primera

El mejor comentario es el que **no hace falta** porque el código ya se explica solo. Un comentario vale la pena cuando explica el **porqué** de una decisión que el código no puede expresar por sí mismo (una regla de negocio rara, una limitación externa, un workaround temporal) — nunca cuando repite lo que la siguiente línea ya dice.

CONTRAEJEMPLO

El comentario que no aporta nada

```
// Suma i más 1  
i = i + 1;  
  
// Recorre la lista de estudiantes  
for (Estudiante e : estudiantes) { ... }
```

Estos comentarios solo repiten lo que el código ya dice con claridad — son ruido, y además hay que mantenerlos sincronizados con el código a mano (y con frecuencia se olvida).

```
// La API del banco exige reintentar hasta 3 veces  
// antes de reportar fallo (ver ticket SOP-114)  
int intentos = 0;  
while (intentos < 3 && !transaccionExitosa) { ... }
```



CONCEPTO

Formato consistente

Un formato consistente reduce la carga mental de leer código ajeno. Java tiene convenciones ampliamente adoptadas (las mismas que ya usas desde la Unidad 0):

Elemento	Convención	Ejemplo
Clases	PascalCase	GestorPedidos
Métodos y variables	camelCase	calcularTotal()
Constantes	MAYUSCULAS_CON_GUION_BAJO	IVA_COLOMBIA
Indentación	Consistente (2 o 4 espacios), nunca mezclada con tabs	—

CONCEPTO

Errores limpios: evita los códigos mágicos

Devolver `-1`, `null` o un código numérico para señalar un error obliga a quien llama el método a recordar y comparar ese valor mágico — y es fácil de olvidar. Java ofrece **excepciones** justamente para esto (se profundiza en la Unidad 4): separan el flujo normal del flujo de error, en vez de mezclarlos con `if` de validación.

```
// Código mágico: fácil de ignorar por accidente
public int buscarEstudiante(String id) {
    if (!existe(id)) return -1;
    // ...
}
```

CONCEPTO

DRY: no te repitas

DRY (*Don't Repeat Yourself*) es el principio de no duplicar la misma lógica en varios lugares del código. La duplicación no es solo más código: es más lugares donde corregir el mismo bug, y es fácil olvidar uno.

```
// Duplicado en tres métodos distintos
double totalConIva1 = precio1 * 1.19;
double totalConIva2 = precio2 * 1.19;
double totalConIva3 = precio3 * 1.19;

// Un solo lugar de verdad
public double aplicarIva(double precio) {
    return precio * 1.19;
}
```

CASO

440 millones de dólares en 45 minutos

En agosto de 2012, la firma financiera **Knight Capital Group** desplegó una actualización de su software de trading a sus servidores de producción. Por un error operativo, un servidor quedó ejecutando código viejo que reutilizaba una bandera (*flag*) que antes activaba una función de prueba llamada "Power Peg" — código que se creía retirado, pero nunca se había eliminado del todo. Esa bandera reactivó lógica obsoleta que empezó a ejecutar órdenes de compra y venta erráticas en el mercado real.

En **45 minutos**, Knight Capital perdió **440 millones de dólares** y la empresa estuvo al borde de la quiebra. La causa raíz, documentada públicamente por la SEC, fue código muerto que nunca se eliminó y una bandera reutilizada sin que nadie recordara para qué servía.

TRANSICIÓN

Higiene de código, no solo estilo

El caso de Knight Capital no fue un problema de sintaxis: fue código sin documentar, sin eliminar, y sin que nadie recordara su propósito. Nombres claros, funciones pequeñas y comentarios que expliquen el porqué no son un capricho estético — son la diferencia entre un equipo que entiende su propio sistema y uno que ya no puede confiar en él.

TENSIÓN CONCEPTUAL

¿Legibilidad o rendimiento?

A veces una versión más "limpia" del código (más métodos pequeños, más abstracciones) tiene un costo mínimo de rendimiento frente a una versión más directa pero menos legible. En la inmensa mayoría de proyectos de curso o de negocio, ese costo es despreciable frente a la ganancia en mantenibilidad — pero vale la pena tenerlo presente: en la próxima sesión se habla de cuándo optimizar sí importa, y cuándo es prematuro preocuparse por eso.

INTERACCIÓN

Refactoricen esta función

En parejas: reescriban esta función aplicando nombres claros y separando responsabilidades.

```
public void p(String s, int t, boolean b) {  
    if (t == 1 && b) {  
        System.out.println("Enviando: " + s);  
    } else if (t == 2) {  
        System.out.println("Guardando: " + s);  
    }  
}
```

6 minutos · compartan su versión con otra pareja

SÍNTESIS

Lo que hay que llevarse de hoy

1. Un nombre bien elegido no necesita un comentario al lado para explicarse.
2. Las funciones deben ser pequeñas, hacer una sola cosa, y tener pocos parámetros.
3. Comenta el **porqué**, no el **qué** — el código ya dice el qué.
4. El formato consistente (PascalCase, camelCase, indentación) reduce la carga de leer código ajeno.
5. Evita código muerto y duplicado: ambos fueron protagonistas de la pérdida de Knight Capital.



CIERRE

Antes de la próxima sesión

- Aplicar al menos una regla de esta sesión (nombres, funciones o comentarios) a un proyecto propio de la Unidad 2.
- Crear una cuenta de GitHub si aún no la tienes — se usa a partir de la próxima sesión.

Próxima sesión: GitHub colaborativo y mejores prácticas — control de versiones, ramas, Pull Requests y code review.

